Exp 9 (version 2)- Model Deployment Using Flask REST API and Docker

Aim

To train a machine learning model, deploy it using a Flask REST API, visualize the results, and prepare the application for Docker containerization.

Procedure

1. Import the required Python libraries and load the Iris dataset.
2. Split the dataset into training and testing sets.
3. Train a Random Forest classification model and save it using Joblib.
4. Create a Flask REST API to accept input features and return predictions in JSON format.
5. Test the API and display the prediction output.
6. Visualize the dataset, confusion matrix, and prediction probabilities using Matplotlib.
7. Create a Dockerfile and requirements file for containerization.
8. Display the results and verify that the deployment files are created successfully.

Code

```
!pip install flask scikit-learn joblib requests matplotlib -q


import os, socket, threading, time, joblib, requests
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from flask import Flask, request, jsonify
from werkzeug.serving import make_server
```

```
iris = load_iris()
X, y = iris.data, iris.target

X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42
)

model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)
joblib.dump(model, "model.pkl")

print("Model trained successfully!")
print("Accuracy:", round(model.score(X_test, y_test) * 100, 2), "%")

app = Flask(__name__)
classes = iris.target_names.tolist()

@app.route("/")
def home():
   return jsonify({"message": "ML Model API is running"})

@app.route("/predict", methods=["POST"])
def predict():
   try:
      features = request.get_json()["features"]
```

```
        if len(features) != 4:
            return jsonify({"error": "Enter 4 features"}), 400
        prediction = int(model.predict([features])[0])
        return jsonify({
            "prediction": classes[prediction],
            "class_id": prediction,
            "probabilities": model.predict_proba([features])[0].tolist()
        })
    except Exception as e:
        return jsonify({"error": str(e)}), 400

sock = socket.socket()
sock.bind(("127.0.0.1", 0))
port = sock.getsockname()[1]
sock.close()

server = make_server("127.0.0.1", port, app)
thread = threading.Thread(target=server.serve_forever, daemon=True)
thread.start()
time.sleep(1)

url = f"http://127.0.0.1:{port}"
print("\nAPI Response:", requests.get(url).json())

sample = [5.1, 3.5, 1.4, 0.2]
response = requests.post(url + "/predict", json={"features": sample})
```

```
result = response.json()

print("\nInput:", sample)
print("Prediction:", result["prediction"])
print("Status Code:", response.status_code)

plt.figure(figsize=(7, 4))
plt.scatter(X_test[:, 0], X_test[:, 1], c=y_test, cmap="viridis")
plt.xlabel("Sepal Length")
plt.ylabel("Sepal Width")
plt.title("Iris Dataset Visualization")
plt.show()

y_pred = model.predict(X_test)
ConfusionMatrixDisplay(
   confusion_matrix(y_test, y_pred),
   display_labels=classes
).plot(cmap="Blues")
plt.title("Confusion Matrix")
plt.show()

plt.figure(figsize=(7, 4))
plt.bar(classes, result["probabilities"])
plt.xlabel("Flower Class")
plt.ylabel("Probability")
plt.title("Prediction Probabilities")
```

```
plt.ylim(0, 1)
plt.show()

with open("requirements.txt", "w") as f:
    f.write("flask\nscikit-learn\njoblib\n")

with open("Dockerfile", "w") as f:
   f.write("""FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY model.pkl .
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
""")

with open("app.py", "w") as f:
   f.write('''from flask import Flask, request, jsonify
import joblib
app = Flask(__name__)
model = joblib.load("model.pkl")
classes = ["setosa", "versicolor", "virginica"]
@app.route("/")
def home():
   return jsonify({"message": "ML Model API is running"})
```

```
@app.route("/predict", methods=["POST"])
def predict():
    try:
        features = request.get_json()["features"]
        if len(features) != 4:
            return jsonify({"error": "Enter 4 features"}), 400
        p = int(model.predict([features])[0])
        return jsonify({"prediction": classes[p], "class_id": p})
    except Exception as e:
        return jsonify({"error": str(e)}), 400
if __name__== "__main__":
    app.run(host="0.0.0.0", port=5000)
''')

print("\nDockerfile created:", os.path.exists("Dockerfile"))
print("Requirements created:", os.path.exists("requirements.txt"))
print("Flask application created:", os.path.exists("app.py"))
print("Model saved:", os.path.exists("model.pkl"))
print("\nDocker commands:")
print("docker build -t ml-model-api .")
print("docker run -p 5000:5000 ml-model-api")

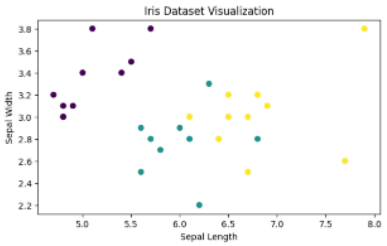
server.shutdown()
print("\nExperiment completed successfully!")
```

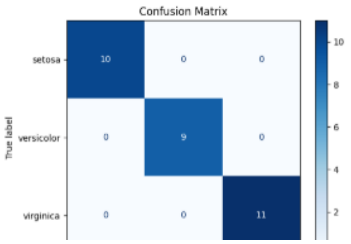
OUTPUT:

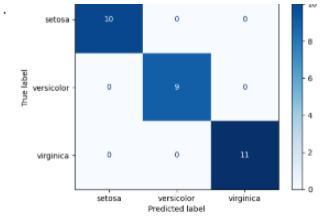
```
Model trained successfully!
Accuracy: 100.0 %
INFO:werkzeug:127.0.0.1 - - [12/Oct/2026 16:53:26] "GET / HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [12/Oct/2026 16:53:26] "POST /predict HTTP/1.1" 200 -

API Home Response:
{'message': 'Iris Prediction API is running'}

Input Features: [5.1, 3.5, 1.4, 0.2]
Predicted Flower: setosa
Probabilities: [0.97984268081632894, 0.020957142708309565, 1.5912840096876760-07]
Status Code: 200
```










```
Files created:
Model: True
Flask application: True
Dockerfile: True
Requirements: True

Docker commands to run in a Docker-enabled environment:
docker build -t iris-api .
docker run -p 5000:5000 iris-api
```

Result

The Random Forest machine learning model was trained and saved successfully. The Flask REST API was tested and returned predictions in JSON format. Graphs and a confusion matrix were displayed, and the Dockerfile and requirements file were created for containerization.